Specifying and Verifying the NOVA Microhypervisor in Concurrent Separation Logic

Hoang-Hai Dang
BlueRock Security, Inc
Germany
hai@bluerock.io

Gregory Malecha
Skylabs AI
USA
gregory@skylabs-ai.com

Abstract

We present the current progress of specifying and verifying the NOVA microhypervisor at BlueRock Security. We carry out these tasks using the BRiCk C++ program logic and verification tools, building on top of the Iris framework for concurrent separation logics (CSLs), in Rocq. We found that the use of CSLs has enabled us to carry out highly modular specifications and verifications of even sophisticated, fine-grained concurrent algorithms in NOVA. We believe this is a significant step forward over other hypervisor verification projects that are either limited to single-threaded verification or are highly non-modular.

Keywords: Iris, separation logic, NOVA, hypervisor

1 Introduction

NOVA [8, 9] is a microhypervisor that provides basic services for virtualization, isolation, scheduling, and management of physical resources. NOVA's microkernel-based design provides highly-efficient, low-level mechanisms and leaves policy decisions and higher-level functionality to be developed on top of it in user mode. BlueRock Security aims to build a modern, trustworthy computing stack atop NOVA, backed by formal proofs.

Background on NOVA. Following the microkernel design philosophy, NOVA only takes control of critical system resources and re-exposes them to clients under a restricted ABI with 16 hypercalls centered around 6 kernel object types.

- A Protection Domain (PD) is a unit of protection and isolation for kernel objects. A PD manages the capabilities to create and control other kernel objects through Object (OBJ) spaces. Other PD spaces exist, *e.g.*, memory spaces (HST, GST) and DMA spaces manage the capabilities to page tables for main memory and devices, respectively.
- Execution Contexts (ECs) and Scheduling Contexts (SCs) execute and schedule user-mode code.
- Portals (PTs) provide user-mode, synchronous, intra-core, inter-process communication.
- Semaphores (SMs) are provide inter-core synchronization and are used to configure interrupts.
- Device Contexts (DCs) are used to track configurations of devices, in relation with interrupt controllers and input-output controllers (SMMUs/IOMMUs).

These kernel objects are exposed through capabilities that include fine-grained permissions and can be delegated between spaces to enable highly modular, least privilege designs. The hypercalls allow user-mode programs to concurrently create and interact with kernel objects and to configure hardware. Other non-critical resources, such as the CPUs running in user mode, user memory, and non-criticial devices are passed through by NOVA and hence are controlled directly by user code. Consequently, a user application running atop NOVA sees that the semantics of physical machine cores (in user mode) extended with the NOVA hypercalls.

Approach. We are following a "spike-driven" verification approach that works down from the formal specification of the hypercall (§3) into the core components that the implementation touches (§4). This approach grounds the proof against the formal spec, which has been helpful to identify subtle concurrency issues that arise, and to force ourselves to adjust either the spec or the implementation appropriately.

The formal spec (`nova_interface`) covers the application binary interface (ABI) of NOVA, using advanced features (*e.g.*, *logical atomicity*) of concurrent separation logics (CSLs), provided by the Iris framework [4, 5]. Meanwhile, the formal proof (`nova_proof`) covers the specs for internal C++ structures and functions and proves that the implementation satisfies those internal specs. The internal specs for the kernel entry points align with the hypercall specs from `nova_interface`. The internal specs and proofs use the BRiCk program logic [6, 11] to reason about the C++ code and rely heavily on advanced CSL features such as ghost state and invariants. We carry out the proofs using SkyLabs' generic separation logic automation and its Rocq-based verification infrastructure [10].

Rocq Artifact. The spec and proof of NOVA, like the implementation itself, are open-sourced and available at https://gitlab.com/bluerocksec/NOVA/-/tree/proof.

2 The Current Progress

The current progress is summarized in Table 1. At the time of writing, the spec covers 8 hypercalls fully and partially covers another 6. The proofs cover 4 hypercalls fully and

RocqPL, Rennes, France
2018. ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

Table 1. Current coverage of the NOVA specs and proofs.

	Hypercall	nova_interface			nova_proof	
		covered	partially covered	not yet covered	verified	not yet verified
1	ipc_call	✓				x
2	ipc_reply	✓				x
3	create_pd	for PDs	for spaces		for PDs	for spaces
4	create_ec	✓				x
5	create_sc	✓			✓	
6	create_pt	✓			✓	
7	create_sm	for user SM	for interrupt SM		for user SM	for interrupt SM
8	create_dc			x		x
9	ctrl_pd		for OBJ spaces	for non-OBJ spaces	for OBJ spaces	for non-OBJ spaces
10	ctrl_ec	✓				x
11	ctrl_sc	✓			✓	
12	ctrl_pt	✓			✓	
13	ctrl_sm	for user SM	for interrupt SM		for user SM	for interrupt SM
14	ctrl_hw			x		x
15	assign_dev		✓			x
16	assign_int		✓			x

partially covers another 4. We note that various internal specs and proofs of the NOVA implementation are not reflected in this table, but will be reused for the proofs of the remaining hypercalls. **The spec and proof are constantly maintained to be up-to-date with the latest release of NOVA (currently August 2025).**

Limitations. The current limitations include:

1. The specs and proofs assume a memory model with sequential consistency, but try to be compatible with weak memory and speculation.
2. We have focused on the architecture-independent portions of NOVA to reduce overhead of hardware modeling, which continues to be a significant challenge.
3. The current effort focuses on C++ code, and not assembly code and its interaction with C++, which is also architecture-specific.

3 The Formal Spec `nova_interface`

We built `nova_interface` as a *hybrid specification* [2, 7] that combines a low-privileged (user-mode) operational CPU model (similar to [1]) with separation logic specs of high-privileged (kernel-mode) behaviors provided by NOVA [3]. In those reports we explain how the spec supports *arbitrary* user code including untrusted program, how clients can use the spec to verify their user programs running atop NOVA, and how one would use the spec to prove higher-level properties such as whole-machine refinements or robust safety.

4 The Formal Proof `nova_proof`

We note several interesting challenges in the proofs.

Hypercall Verifications. The main difficulty in verifying hypercalls is aligning the linearization points of the specification with the implementation. For example, `ctrl_pt`'s spec has 3 linearization points, but the implementation takes 7 steps, some of which are fine-grained atomics.

The Object Soup. Graph data structures are difficult to model in CSLs due to arbitrary aliasing. NOVA's core state is a collection of inter-linked kernel objects that we refer to as the "object soup". To model the soup, we indirect the ownership through *object names* (IDs) and *reference tokens*. The tokens support 3 types of pointers defined by NOVA to mediate the lifetime of objects and their access: reference-counted pointers, internal pointers, and transient pointers.

Scheduler Verification. At the high-level, NOVA scheduling appears relatively simple because schedulers are intra-core and threads (ECs) cannot migrate between cores. However, the difficulty arises due to semaphores (SMs), which provide cross-core communication. When an EC on one core blocks on an SM, its scheduling context (SC) is moved from the scheduler's ready queue to the SM's blocked queue. When the SM is upped on another core, that core sends the SC back to the scheduler on the other core. Our proof models this protocol, including NOVA's subtle exploitation of the fact that there is always some SC to be executed.

5 Conclusion

Our work on the NOVA proof highlights a number of challenges in scaling formal methods to mainstream software, but suggests that CSL is well-suited to this challenge.

References

[1] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Shaked Flur, Kathryn E. Gray, Prashanth Mundkur, Robert M. Norton, Christopher Pulte, Alastair Reid, Peter Sewell, Ian Stark, and Mark Wassell. 2018. Detailed Models of Instruction Set Architectures: From Pseudocode to Formal Semantics. In *Proc. Automated Reasoning Workshop*. 23–24. Two-page abstract.

[2] Hoang-Hai Dang, David Swasey, Paolo Giarrusso, and Gregory Malecha. 2024. *A Formal Specification of the NOVA Microhypervisor's ABI.* Technical Report. BlueRock Security, Inc. https://bluerocksec.gitlab.io/formal-methods/tech_reports/a-formal-specification-of-the-nova-microhypervisor-abi/

[3] Paolo G. Giarrusso, Abhishek Anand, Gregory Malecha, František Farka, and Hoang-Hai Dang. 2024. Modularizing CPU Semantics for Virtualization, Talk at PriSC 2024. https://popl24.sigplan.org/details/prisc-2024-papers/7/Modularizing-CPU-Semantics-for-Virtualization

[4] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20. doi:10.1017/S0956796818000151

[5] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. 2015. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*. ACM, 637–650. doi:10.1145/2676726.2676980

[6] Gregory Malecha, Abhishek Anand, and Gordon Stewart. 2020. Towards an Axomatic Basis for C++. In *The 11th Coq Workshop* (online).

[7] Gregory Malecha, Hoang-Hai Dang, Paolo G. Giarrusso, Simon Hudon, Jan-Oliver Kaiser, and David Swasey. 2025. Modular, Full-System Verification. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems* (Banff, AB, Canada) *(HotOS '25)*. Association for Computing Machinery, New York, NY, USA, 42–49. doi:10.1145/3713082.3730387

[8] Udo Steinberg. 2025. NOVA Microhypervisor Interface Specification (August 13, 2025). https://gitlab.com/bluerocksec/NOVA/-/blob/9d7ae38a0dc8605cdb3253512cff8e1af7621e1c/doc/specification.pdf

[9] Udo Steinberg and Bernhard Kauer. 2010. NOVA: A Microhypervisor-Based Secure Virtualization Architecture. In *Proceedings of the 5th European Conference on Computer Systems* (Paris, France) *(EuroSys '10)*. Association for Computing Machinery, New York, NY, USA, 209–222. doi:10.1145/1755913.1755935

[10] The SkyLabs AI Formal Methods Team. 2025. SkyLabs AI: Formal Methods. https://skylabsai.github.io/auto-docs/

[11] The SkyLabs AI Formal Methods Team. 2025. The BRiCk C++ Program Logic. https://skylabsai.github.io/BRiCk/